The Ultimate Recursion Reference Guide

From First Principles to Advanced Patterns

Sahil Raj (After Dark)

Contents

Chapter 1

Fundamentals of Recursion

1.1 What Is Recursion?

Pattern / Core Idea

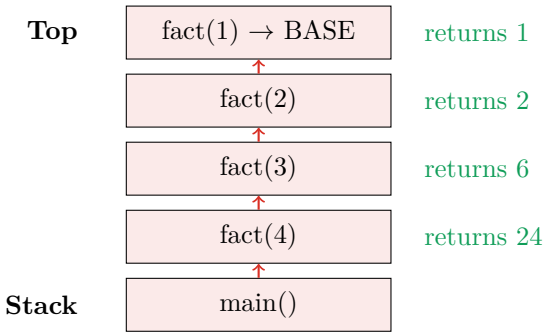
A function is **recursive** if it calls itself to solve a smaller version of the same problem.
Every recursive solution has two mandatory parts:

1. **Base case:** The smallest input where the answer is trivially known (no recursive call).
2. **Recursive case:** Break the problem into one or more strictly smaller subproblems, solve them, then combine.

Key invariant: Each recursive call must make *progress* toward the base case; otherwise, you get infinite recursion.

1.1.1 The Call Stack

Every function call allocates a *stack frame* storing local variables and the return address. Recursion stacks these frames. The maximum depth before stack overflow is typically $\sim 10^4$–10^5 frames on most systems.



1.2 Template for Every Recursive Function

```
ReturnType solve(Parameters params) {
    // 1. Base case(s) -- ALWAYS first
    if (base_condition) return base_value;

    // 2. Recursive case -- make problem smaller
    SubResult r = solve(smaller_params);

    // 3. Combine and return
    return combine(r, current_element);
}
```

Common Trap / Gotcha

Forgetting the base case is the #1 recursion bug. Always write it first. If your function can receive an empty array, negative index, or zero, handle those explicitly.

1.3 Factorial & Fibonacci — Building Intuition

1.3.1 Factorial

$n! = n \times (n-1)!$, with $0! = 1$.

```
long long factorial(int n) {
    if (n <= 0) return 1;           // base case
    return n * factorial(n - 1);    // recursive case
}
```

Complexity

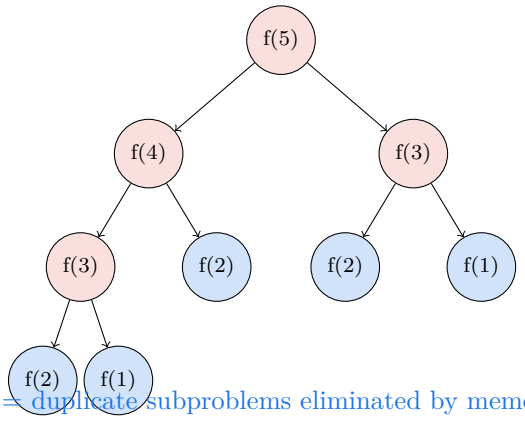
$\mathcal{O}(n)$ time, $\mathcal{O}(n)$ stack space (one frame per call).

1.3.2 Fibonacci (Naive vs Memoized)

```
// Naive: exponential -- DO NOT USE for large n
int fib_naive(int n) {
    if (n <= 1) return n;
    return fib_naive(n - 1) + fib_naive(n - 2);  // O(2^n)
}

// Memoized: linear -- correct approach
int memo[100005];
int fib_memo(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib_memo(n-1) + fib_memo(n-2);
}
```

Key Insight / Optimization

The memoization pattern: Check cache → compute if missing → store → return. This converts exponential naive recursion to linear. Fibonacci has only n unique subproblems despite 2^n call-tree nodes naively.



Red = duplicate subproblems eliminated by memoization

1.4 Recursion on Arrays

1.4.1 Sum of Array

```cpp
int sumArr(vector<int>& arr, int l, int r) {
    if (l > r) return 0;
    return arr[l] + sumArr(arr, l + 1, r);
}
```

1.4.2 Maximum Element

```cpp
int maxArr(vector<int>& arr, int i) {
    if (i == (int)arr.size() - 1) return arr[i];  // base: last element
    return max(arr[i], maxArr(arr, i + 1));          // max of first vs rest
}
```

1.4.3 Count Occurrences of an Element

```cpp
int countOccurrences(vector<int>& arr, int i, int target) {
    if (i == (int)arr.size()) return 0;
    return (arr[i] == target ? 1 : 0) + countOccurrences(arr, i + 1, target);
}
```

1.4.4 Check if Array is Sorted

```cpp
bool isSorted(vector<int>& arr, int i) {
    if (i >= (int)arr.size() - 1) return true;
    if (arr[i] > arr[i + 1]) return false;
    return isSorted(arr, i + 1);
}
```

1.4.5 Reverse an Array In-Place

```cpp
void reverseArr(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    swap(arr[l], arr[r]);
    reverseArr(arr, l + 1, r - 1);
}
```

1.4.6 Move All Zeros to End

```cpp
// Returns index of next position to place non-zero element
int moveZeros(vector<int>& arr, int i, int j) {
    if (i == (int)arr.size()) {
        // Fill remaining with zeros
        while (j < i) arr[j++] = 0;
        return j;
    }
    if (arr[i] != 0) arr[j++] = arr[i];
    return moveZeros(arr, i + 1, j);
}
// Call: moveZeros(arr, 0, 0)
```

Complexity

All array recursions above: $\mathcal{O}(n)$ time, $\mathcal{O}(n)$ stack space.

1.5 Head vs Tail Recursion

Pattern / Core Idea

Head recursion: The recursive call comes *before* any work. Processing happens on the way *back up* (unwinding).
Tail recursion: The recursive call is the *last* thing done. Work happens on the way *down*. Modern compilers can optimize this into a loop (*tail call optimization*).

```cpp
// Head recursion: prints 1 2 3 4 5 (work done on the way back up)
void headPrint(int n) {
    if (n == 0) return;
    headPrint(n - 1);   // recurse FIRST
    cout << n << " ";   // print AFTER return
}

// Tail recursion: prints 5 4 3 2 1 (work done on the way down)
void tailPrint(int n) {
    if (n == 0) return;
    cout << n << " ";   // print FIRST
    tailPrint(n - 1);   // recurse AFTER
}

// Tail-recursive factorial with accumulator
long long factTail(int n, long long acc = 1) {
    if (n <= 0) return acc;
    return factTail(n - 1, acc * n);   // last operation is the call
}
```

Key Insight / Optimization

Converting tail recursion to iteration: Any tail-recursive function maps directly to a `while` loop. The accumulator becomes a loop variable. GCC/Clang optimize tail calls with `-O2`.

1.6 Multiple Base Cases & Multiple Recursive Calls

Pattern / Core Idea

Not all problems have a single base case or a single recursive call. Match the number of base cases to the number of "boundary" inputs, and the number of recursive calls to the number of subproblems you need to combine.

1.6.1 Staircase Problem (1, 2, or 3 Steps)

```cpp
// Count ways to climb n stairs taking 1, 2, or 3 steps at a time
int climbStairs(int n) {
    if (n < 0) return 0;        // invalid: overshot
    if (n == 0) return 1;       // base: exactly at top (1 way)
    return climbStairs(n - 1) + climbStairs(n - 2) + climbStairs(n - 3);
}

// With memoization (required for correctness at scale)
int memo[10005];
int climbMemo(int n) {
    if (n < 0) return 0;
    if (n == 0) return 1;
    if (memo[n] != -1) return memo[n];
    return memo[n] = climbMemo(n-1) + climbMemo(n-2) + climbMemo(n-3);
}
```

1.6.2 Jump Game: Can You Reach the End?

From index i, you can jump up to `arr[i]` steps forward. Can you reach the last index?

```cpp
bool canJump(vector<int>& arr, int i) {
    if (i >= (int)arr.size() - 1) return true;    // reached or passed end
    if (arr[i] == 0) return false;                 // stuck
    for (int step = 1; step <= arr[i]; step++)
        if (canJump(arr, i + step)) return true;
    return false;
}
```

Common Trap / Gotcha

Without memoization, this is $\mathcal{O}(n^2)$ in the worst case and visits many redundant states. Add a `memo[]` array to cache the result for each index — this makes it $\mathcal{O}(n)$.

1.7 Recursion and the "Think Small" Mindset

When approaching a new problem recursively, ask three questions:

1. **What is the smallest/simplest version of this problem?** → Write the base case.
2. **If I already had the answer for a slightly smaller input, how would I use it?** → Write the recurrence.
3. **Am I definitely making the problem smaller?** → Verify progress.

1.7.1 Example: Print Numbers from 1 to N (Both Ways)

```cpp
// Ascending: 1 2 3 ... n
void printAsc(int i, int n) {
    if (i > n) return;
    cout << i << " ";
    printAsc(i + 1, n);
}

// Descending: n n-1 ... 1
void printDesc(int n) {
    if (n < 1) return;
    cout << n << " ";
    printDesc(n - 1);
}

// Both in one function using head recursion trick
void printBoth(int i, int n) {
    if (i > n) return;
    cout << i << " ";        // prints 1 2 3 ... n going down
    printBoth(i + 1, n);
    cout << i << " ";        // prints n ... 2 1 coming back up
}
```

Chapter 2

Classic Recursive Problems

2.1 Binary Search and Variations

Pattern / Core Idea

Idea: At each step, eliminate half the search space by comparing with the midpoint.
State: `search(arr, lo, hi, target)`
Base cases: `lo > hi` (not found) or `arr[mid] == target` (found).

2.1.1 Standard Binary Search

```
int binarySearch(vector<int>& arr, int lo, int hi, int target) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return binarySearch(arr, mid + 1, hi, target);
    return binarySearch(arr, lo, mid - 1, target);
}
```

Complexity

$\mathcal{O}(\log n)$ time, $\mathcal{O}(\log n)$ stack. Iterative version uses $\mathcal{O}(1)$ space.

Common Trap / Gotcha

Integer overflow: Never compute `mid = (lo + hi) / 2`. Use `lo + (hi - lo) / 2` to avoid overflow when `lo + hi > INT_MAX`.

2.1.2 Variation: First and Last Occurrence

```
// First occurrence of target (-1 if not found)
int firstOccurrence(vector<int>& arr, int lo, int hi, int target) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) {
        int left = firstOccurrence(arr, lo, mid - 1, target);
        return (left == -1) ? mid : left;   // check if there's an earlier one
    }
    if (arr[mid] < target) return firstOccurrence(arr, mid + 1, hi, target);
    return firstOccurrence(arr, lo, mid - 1, target);
}

// Last occurrence of target (-1 if not found)
```

```cpp
int lastOccurrence(vector<int>& arr, int lo, int hi, int target) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) {
        int right = lastOccurrence(arr, mid + 1, hi, target);
        return (right == -1) ? mid : right;
    }
    if (arr[mid] < target) return lastOccurrence(arr, mid + 1, hi, target);
    return lastOccurrence(arr, lo, mid - 1, target);
}
```

2.1.3 Variation: Search in Rotated Sorted Array (LeetCode 33)

```cpp
int searchRotated(vector<int>& arr, int lo, int hi, int target) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;

    if (arr[lo] <= arr[mid]) {              // left half is sorted
        if (arr[lo] <= target && target < arr[mid])
            return searchRotated(arr, lo, mid - 1, target);
        return searchRotated(arr, mid + 1, hi, target);
    } else {                                // right half is sorted
        if (arr[mid] < target && target <= arr[hi])
            return searchRotated(arr, mid + 1, hi, target);
        return searchRotated(arr, lo, mid - 1, target);
    }
}
```

2.2 Power Function (Fast Exponentiation)

Pattern / Core Idea

$x^n = (x^{n/2})^2$ for even n; $x^n = x \cdot x^{n-1}$ for odd n. Halves the problem each time.

```cpp
long long power(long long base, long long exp, long long mod) {
    if (exp == 0) return 1;
    if (exp % 2 == 0) {
        long long half = power(base, exp / 2, mod);
        return (half * half) % mod;
    }
    return (base % mod * power(base, exp - 1, mod)) % mod;
}
```

Complexity

$\mathcal{O}(\log n)$ time and stack space.

2.3 Tower of Hanoi

Pattern / Core Idea

Move n disks from peg `src` to peg `dst` using `aux`.
Key insight: Move $n-1$ disks to `aux`, move the largest to `dst`, then move $n-1$ disks from `aux` to `dst`.

```cpp
void hanoi(int n, char src, char dst, char aux) {
    if (n == 0) return;
```

```
    hanoi(n - 1, src, aux, dst);
    cout << "Move disk " << n << " from " << src << " to " << dst << "\n";
    hanoi(n - 1, aux, dst, src);
}
// Minimum moves = 2^n - 1
```

Complexity

$\mathcal{O}(2^n)$ time. The minimum number of moves is exactly $2^n - 1$. $\mathcal{O}(n)$ stack space.

2.4 Tower of Hanoi — Variations

2.4.1 Variation 1: Count Minimum Moves Only

```
// Returns the minimum number of moves to transfer n disks
long long hanoiCount(int n) {
    if (n == 0) return 0;
    return 2 * hanoiCount(n - 1) + 1;   // recurrence: T(n) = 2*T(n-1) + 1
}
// Closed form: 2^n - 1
// T(1)=1, T(2)=3, T(3)=7, T(4)=15 ...
```

2.4.2 Variation 2: Hanoi with Forbidden Move (Cyclic Pegs)

Pegs are arranged in a cycle: A → B → C → A. You cannot move directly from A to C (or C to A). You must always move to the adjacent peg.

```
// Moves n disks from src to dst, going through intermediate pegs
// direct = true means we can use the direct path, false means go the long way
long long hanoiForbidden(int n, char src, char dst, char aux) {
    // In cyclic Hanoi: if src and dst are non-adjacent, route through aux
    // The recurrence becomes T(n) = 3*T(n-1) + 2, giving T(n) = 3^n - 1 moves
    if (n == 0) return 0;
    // Move n-1 to dst (going the "wrong" way around the cycle, so costs more)
    long long a = hanoiForbidden(n - 1, src, dst, aux);
    // Move disk n from src to aux
    cout << "Move disk " << n << " from " << src << " to " << aux << "\n";
    // Move n-1 from dst back to src
    long long b = hanoiForbidden(n - 1, dst, src, aux);
    // Move disk n from aux to dst
    cout << "Move disk " << n << " from " << aux << " to " << dst << "\n";
    // Move n-1 from src to dst
    long long c = hanoiForbidden(n - 1, src, dst, aux);
    return a + 1 + b + 1 + c;
}
// Minimum moves = 3^n - 1
```

Complexity

$T(n) = 3T(n-1) + 2 \Rightarrow T(n) = 3^n - 1$ moves. The forbidden constraint triples the work!

2.4.3 Variation 3: Frame-Stewart (4 Pegs)

With 4 pegs, the Frame-Stewart conjecture gives the optimal solution:

```
// Optimal 4-peg Hanoi (Frame-Stewart conjecture)
// Move k disks to spare peg, then 3-peg Hanoi for remaining n-k, then move k back
// We try all values of k from 1 to n-1 and pick the minimum
int hanoi4Peg(int n) {
    if (n <= 0) return 0;
```

```
    if (n == 1) return 1;
    if (n == 2) return 3;

    int minMoves = INT_MAX;
    for (int k = 1; k < n; k++) {
        // Move k disks with 4 pegs, then n-k disks with 3 pegs, then k disks with 4 pegs
        int moves = 2 * hanoi4Peg(k) + (1 << (n - k)) - 1;
        minMoves = min(minMoves, moves);
    }
    return minMoves;
}
```

Key Insight / Optimization

Key insight: With an extra peg, we partition the n disks: move the top k disks (using all 4 pegs), then move the bottom $n-k$ disks to the destination (using only 3 pegs, ignoring the temporarily occupied 4th), then move the k disks onto the stack.

2.5 Tower of Hanoi — Magical Cake (Generalized)

Classic Problem

Alice has n magical cake layers. Layer i (layer 1 = smallest on top, layer n = largest at the bottom) is movable only when there are *exactly* a_i layers above it at its current peg. Move all layers from peg 1 to peg 3 in at most 2^n moves, or report `NO`.
Contrast with standard TOH: In standard TOH $a_i = i - 1$: disk i can move only when all $i-1$ smaller disks are stacked on top of it. Here, each layer has its own custom unlock count, making this a true generalization of TOH.

2.5.1 Step 1: Feasibility Check

Layer i (1-indexed) has at most $i-1$ layers above it, so we need $a_i \le i-1$. In 0-indexed code `v[i]` $= a_{i+1}$, so `v[i] >= i+1` catches any impossible input:

```
for (int i = 0; i < n; i++) {
    if (v[i] >= i + 1) {    // layer i+1 can have at most i layers above it
        cout << "No\n";     // but needs v[i] >= i+1: structurally impossible
        return;
    }
}
```

2.5.2 Step 2: State Representation

```
struct bagga {
    int start, helper, end;
    // One recorded move: layer `start` moves FROM peg `helper` TO peg `end`
};

vector<int> ind(n + 1, 1);  // ind[i] = current peg of layer i (all start on peg 1)
vector<bagga> tot;           // the complete sequence of moves to print
```

Key Insight / Optimization

The "6 trick": Pegs are numbered 1, 2, 3 and $1+2+3=6$. Given any two distinct pegs a and b, the third peg is always $6-a-b$. This eliminates every if/else when looking up the auxiliary peg — a classic competitive programming micro-trick.

```
// Current peg 1, destination peg 3 --> helper = 6 - 1 - 3 = 2  (correct!)
// Current peg 2, destination peg 3 --> helper = 6 - 2 - 3 = 1  (correct!)
// Current peg 1, destination peg 2 --> helper = 6 - 1 - 2 = 3  (correct!)
```

2.5.3 Step 3: The Core Recursive Function

The function `recur(index, source, help, endd)` moves layer `index` from peg `source` to peg `endd`, using `help` as the auxiliary.

```
auto recur = [&](auto&& f, int index, int source, int help, int endd) -> void {

    // Early exit: layer is already at the destination -- nothing to do.
    if (ind[index] == endd) return;

    // ================================================================
    // PHASE 1 -- Bring the "lock layers" onto source.
    //
    // Layer `index` needs exactly v[index-1] layers above it to be movable.
    // The v[index-1] layers with indices (index-1) down to (index-v[index-1])
    // must be sitting on `source` on top of `index`.
    // Move each of them there recursively (they might be on other pegs).
    //
    //   j iterates from index-1 DOWN to index-v[index-1]
    //   For each j:  destination = source,  helper = 6 - ind[j] - source
    // ================================================================
    for (int j = index - 1; j >= index - v[index - 1]; j--)
        f(f, j, ind[j], 6 - ind[j] - source, source);

    // ================================================================
    // PHASE 2 -- Clear the excess layers away from source.
    //
    // Layers 1 to (index-1-v[index-1]) must NOT be above `index` on source
    // (that would make the count exceed a[index]).
    // Move each of them to `help` (the out-of-the-way peg).
    //
    //   j iterates from (index-1-v[index-1]) DOWN to 1
    //   For each j:  destination = help,  helper = 6 - ind[j] - help
    // ================================================================
    for (int j = index - 1 - v[index - 1]; j >= 1; j--)
        f(f, j, ind[j], 6 - ind[j] - help, help);

    // ================================================================
    // PHASE 3 -- The unlock condition is now satisfied.
    //
    // Layer `index` has exactly v[index-1] layers above it on `source`.
    // It is movable. Record the move and update its position.
    // ================================================================
    tot.push_back({index, source, endd});
    ind[index] = endd;
};
```

Pattern / Core Idea

Why this is TOH on steroids: Standard TOH recursion has one job: clear $n-1$ layers, move the biggest, restore $n-1$ layers. Here the unlock condition a_i varies per layer, so "clearing" and "restoring" layers requires splitting them into two groups (Phases 1 & 2) instead of treating them uniformly. The elegance is that the three-line `source/help/endd` parametrization already encodes which peg is which — the algorithm generalizes naturally.

2.5.4 Step 4: The Outer Loop

```
// Process layers from largest (n) to smallest (1).
// `3 - ind[i]` gives the third peg (not ind[i] and not 3).
for (int i = n; i >= 1; --i) {
    recur(recur, i,  ind[i],  3 - ind[i],  3);
    //           ^   ^from    ^aux          ^to peg 3
}
```

Key Insight / Optimization

Why process largest-first? Layer n (the base of the cake) must go to peg 3 first — nothing sits below it, so it's the "free" layer to target. Its `recur` call will internally move any smaller layers it needs. When the loop reaches smaller i, `ind[i]` may already be 3 (moved as a side-effect), so the early-return fires and no duplicate work happens.

2.5.5 Step 5: Full Solution (Your Code)

```
struct bagga { int start, helper, end; };  // one recorded move

void solve() {
    int n; cin >> n;
    vector<int> v(n);
    for (int i = 0; i < n; i++) cin >> v[i];

    // Feasibility: a[i+1] = v[i] must be <= i (layer i+1 has at most i above it)
    for (int i = 0; i < n; i++) {
        if (v[i] >= i + 1) { cout << "No\n"; return; }
    }

    vector<int> ind(n + 1, 1);  // all layers start on peg 1
    vector<bagga> tot;

    auto recur = [&](auto&& f, int index, int source, int help, int endd) -> void {
        if (ind[index] == endd) return;          // already there: skip

        // Phase 1: bring v[index-1] lock-layers to source
        for (int j = index - 1; j >= index - v[index - 1]; j--)
            f(f, j, ind[j], 6 - ind[j] - source, source);

        // Phase 2: move excess layers away to help
        for (int j = index - 1 - v[index - 1]; j >= 1; j--)
            f(f, j, ind[j], 6 - ind[j] - help, help);

        // Phase 3: move layer index
        tot.push_back({index, source, endd});
        ind[index] = endd;
    };

    // Move all layers (largest first) to peg 3
    for (int i = n; i >= 1; --i)
        recur(recur, i, ind[i], 3 - ind[i], 3);

```

```
    cout << "Yes\n" << tot.size() << '\n';
    for (auto& m : tot)
        cout << m.start << " " << m.helper << " " << m.end << '\n';
}
```

2.5.6 Worked Example: $n = 3$, $a = [0, 1, 2]$ (standard TOH)

This is exactly standard TOH. Every layer i needs $i - 1$ layers above it.

Step	Action (layer, from, to)	Peg 1	Peg 2	Peg 3
Initial	—	[1,2,3]	[]	[]
1	Layer 1: peg 1 → 3	[2,3]	[]	[1]
2	Layer 2: peg 1 → 2	[3]	[2]	[1]
3	Layer 1: peg 3 → 2	[3]	[1,2]	[]
4	Layer 3: peg 1 → 3	[]	[1,2]	[3]
5	Layer 1: peg 2 → 1	[1]	[2]	[3]
6	Layer 2: peg 2 → 3	[1]	[]	[2,3]
7	Layer 1: peg 1 → 3	[]	[]	[1,2,3]

Trace of outer loop, $i = 3$ (move layer 3 from peg 1 to peg 3):

- `v[2]=2`: Phase 1 brings layers 2 and 1 to peg 1 (already there: early return).
- `v[2]=2`: Phase 2 moves layers {none} to help (loop runs 0 times since $3 - 1 - 2 = 0$).
- Unlock: 2 layers above layer 3 on peg 1. Move layer 3 to peg 3. (*Step 4*)

Non-trivial example: $n = 3$, $a = [0, 0, 1]$: layer 3 only needs 1 layer above it.

- Phase 1 brings just layer 2 to peg 1. But layer 1 is on top of it — the recursive call for layer 2 runs Phase 2: move layer 1 to help (peg 2). Then layer 2 moves to peg 1 (trivial).
- Phase 2 of the outer call moves layer 1 (the only excess) to help (peg 2).
- Now layer 3 has exactly 1 layer above it on peg 1. Move it to peg 3.

Total: 4 moves instead of 7 — the relaxed unlock condition saves moves!

Complexity

Total moves $\leq 2^n - 1$. For $n \leq 20$ (problem constraint: sum of $2^n \leq 2^{20}$), output fits comfortably. Time and space: $\mathcal{O}(2^n)$.

Common Trap / Gotcha

Phase 1 loop direction: The loop runs *downward* from `index-1` to `index-v[index-1]`. These are the larger-indexed layers in the range above `index`, physically sitting *below* the smaller layers on the peg. Moving the larger-indexed ones first would be inaccessible — but the recursion handles this: each `f(f, j, ...)` call will itself run its own Phase 2 to clear whatever is on top of layer j before moving it. This self-similar structure is what makes the generalized algorithm correct.

2.6 GCD (Euclidean Algorithm) and Variations

2.6.1 Standard GCD

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

2.6.2 Extended GCD (Bezout Coefficients)

Finds x, y such that $ax + by = \gcd(a, b)$.

```
int extGcd(int a, int b, int& x, int& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    int x1, y1;
    int g = extGcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
// Application: modular inverse of a mod m (when gcd(a,m) == 1)
// extGcd(a, m, x, y) => x is the inverse (take x mod m)
```

2.6.3 GCD of an Array

```
int gcdArray(vector<int>& arr, int i) {
    if (i == (int)arr.size() - 1) return arr[i];
    return gcd(arr[i], gcdArray(arr, i + 1));
}
// LCM of array: lcm(a, b) = a / gcd(a, b) * b  (avoid overflow)
int lcmArray(vector<int>& arr, int i) {
    if (i == (int)arr.size() - 1) return arr[i];
    long long l = lcmArray(arr, i + 1);
    return arr[i] / gcd(arr[i], (int)l) * l;
}
```

2.7 String Recursion

2.7.1 Check Palindrome

```
bool isPalindrome(const string& s, int l, int r) {
    if (l >= r) return true;
    if (s[l] != s[r]) return false;
    return isPalindrome(s, l + 1, r - 1);
}
```

2.7.2 Check if String B is a Subsequence of A

```
bool isSubseq(const string& a, const string& b, int i, int j) {
    if (j == (int)b.size()) return true;   // matched all of b
    if (i == (int)a.size()) return false;  // exhausted a
    if (a[i] == b[j]) return isSubseq(a, b, i + 1, j + 1);
    return isSubseq(a, b, i + 1, j);
}
```

2.7.3 All Subsequences Summing to K

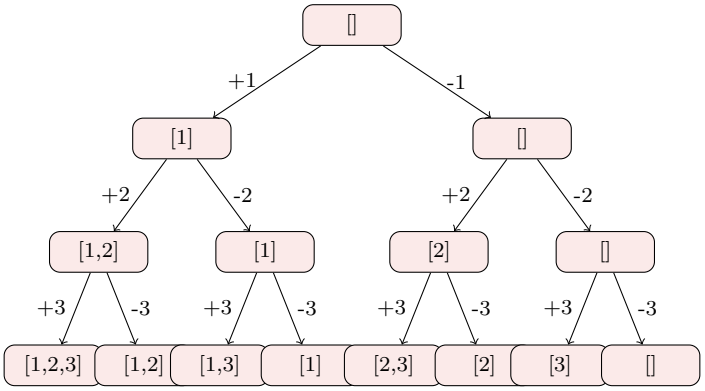
```
void subsetsWithSum(vector<int>& arr, int i, int k,
                    vector<int>& curr, vector<vector<int>>& result) {
    if (k == 0) { result.push_back(curr); return; }
    if (i == (int)arr.size() || k < 0) return;

    // Include arr[i]
    curr.push_back(arr[i]);
    subsetsWithSum(arr, i + 1, k - arr[i], curr, result);
    curr.pop_back();

    // Exclude arr[i]
    subsetsWithSum(arr, i + 1, k, curr, result);
}
```

2.8 Print All Subsequences / Subsets

Pattern / Core Idea

At each index, make a binary choice: *include* or *exclude* the current element. The recursion tree has 2^n leaves, one per subset.

```
void subsets(vector<int>& arr, int i, vector<int>& current) {
    if (i == (int)arr.size()) {
        for (int x : current) cout << x << " ";
        cout << "\n";
        return;
    }
    current.push_back(arr[i]);
    subsets(arr, i + 1, current);     // include
    current.pop_back();
    subsets(arr, i + 1, current);     // exclude
}
```



Complexity

$\mathcal{O}(2^n)$ time (all subsets), $\mathcal{O}(n)$ stack + $\mathcal{O}(n)$ for the current vector.

Chapter 3

Backtracking

3.1 The Backtracking Template

Pattern / Core Idea

Backtracking is **recursion + undo**. Try a choice, recurse, then undo the choice (backtrack) before trying the next one.

Steps:

1. If we've built a complete solution: record it and return.
2. For each valid choice at the current step:
   - Apply the choice (modify state).
   - Recurse to the next step.
   - Undo the choice (restore state).

Pruning: Skip branches that can never lead to a valid solution.

```
void backtrack(State& state, Results& results) {
    if (isComplete(state)) { results.push_back(state); return; }
    for (Choice c : getChoices(state)) {
        if (!isValid(state, c)) continue;  // pruning
        apply(state, c);
        backtrack(state, results);
        undo(state, c);
    }
}
```

3.2 Permutations

3.2.1 All Permutations (Swap Method)

```
void permutations(vector<int>& arr, int start, vector<vector<int>>& result) {
    if (start == (int)arr.size()) { result.push_back(arr); return; }
    for (int i = start; i < (int)arr.size(); i++) {
        swap(arr[start], arr[i]);
        permutations(arr, start + 1, result);
        swap(arr[start], arr[i]);           // undo
    }
}
```

Complexity

$\mathcal{O}(n! \cdot n)$ time to generate and store all permutations. $\mathcal{O}(n)$ stack space.

3.2.2 Permutations with Duplicates (Unique Only)

```cpp
void uniquePerms(vector<int>& arr, int start, vector<vector<int>>& result) {
    if (start == (int)arr.size()) { result.push_back(arr); return; }
    set<int> seen;
    for (int i = start; i < (int)arr.size(); i++) {
        if (seen.count(arr[i])) continue;   // skip duplicate at this position
        seen.insert(arr[i]);
        swap(arr[start], arr[i]);
        uniquePerms(arr, start + 1, result);
        swap(arr[start], arr[i]);
    }
}
```

3.2.3 Next Permutation (Single Step)

```cpp
// Generate next permutation in lexicographic order (modifies in-place)
bool nextPermutation(vector<int>& arr) {
    int n = arr.size(), i = n - 2;
    while (i >= 0 && arr[i] >= arr[i+1]) i--;     // find descent
    if (i < 0) return false;                        // last permutation
    int j = n - 1;
    while (arr[j] <= arr[i]) j--;
    swap(arr[i], arr[j]);
    reverse(arr.begin() + i + 1, arr.end());
    return true;
}
```

3.3 Combinations

3.3.1 Combinations of Size k (LeetCode 77)

```cpp
void combinations(int n, int k, int start, vector<int>& curr,
                  vector<vector<int>>& result) {
    if ((int)curr.size() == k) { result.push_back(curr); return; }
    int remaining = n - start + 1, need = k - (int)curr.size();
    if (remaining < need) return;                 // pruning
    for (int i = start; i <= n; i++) {
        curr.push_back(i);
        combinations(n, k, i + 1, curr, result);
        curr.pop_back();
    }
}
```

3.3.2 Combination Sum (Unlimited Use, LeetCode 39)

```cpp
void combinationSum(vector<int>& cands, int target, int start,
                    vector<int>& curr, vector<vector<int>>& result) {
    if (target == 0) { result.push_back(curr); return; }
    for (int i = start; i < (int)cands.size(); i++) {
        if (cands[i] > target) break;             // sort cands first; prune
        curr.push_back(cands[i]);
        combinationSum(cands, target - cands[i], i, curr, result); // reuse allowed
        curr.pop_back();
    }
}
```

3.3.3 Combination Sum II (Each Number Used Once, LeetCode 40)

```cpp
void combinationSum2(vector<int>& cands, int target, int start,
                     vector<int>& curr, vector<vector<int>>& result) {
    if (target == 0) { result.push_back(curr); return; }
```

```
    for (int i = start; i < (int)cands.size(); i++) {
        if (cands[i] > target) break;
        if (i > start && cands[i] == cands[i-1]) continue; // skip duplicates at same level
        curr.push_back(cands[i]);
        combinationSum2(cands, target - cands[i], i + 1, curr, result); // no reuse
        curr.pop_back();
    }
}
// Call: sort(cands.begin(), cands.end()); combinationSum2(...)
```

3.4 Generate All Parentheses (LeetCode 22)

```
void genParens(int n, int open, int close, string& curr,
               vector<string>& result) {
    if ((int)curr.size() == 2 * n) { result.push_back(curr); return; }
    if (open < n) {                              // can add '('
        curr += '(';
        genParens(n, open + 1, close, curr, result);
        curr.pop_back();
    }
    if (close < open) {                          // can add ')' only if open > close
        curr += ')';
        genParens(n, open, close + 1, curr, result);
        curr.pop_back();
    }
}
// For n=3: "((()))", "(()())", "(())()", "()(())", "()()()"
```

Key Insight / Optimization

Pruning rule: We can only add '(' if we have fewer than n open parens. We can only add ')' if we have more open than closed — this single constraint eliminates all invalid sequences without ever building them.

3.5 Phone Digit Letter Combinations (LeetCode 17)

```
string phoneMap[] = {"", "", "abc", "def", "ghi", "jkl",
                     "mno", "pqrs", "tuv", "wxyz"};

void letterCombinations(const string& digits, int i, string& curr,
                        vector<string>& result) {
    if (i == (int)digits.size()) { result.push_back(curr); return; }
    for (char c : phoneMap[digits[i] - '0']) {
        curr += c;
        letterCombinations(digits, i + 1, curr, result);
        curr.pop_back();
    }
}
// For "23": ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

3.6 N-Queens Problem

```
vector<vector<string>> solveNQueens(int n) {
    vector<vector<string>> result;
    vector<int> queens(n, -1);
    vector<bool> col(n, false), diag1(2*n, false), diag2(2*n, false);

    function<void(int)> backtrack = [&](int row) {
        if (row == n) {
            vector<string> board(n, string(n, '.'));
```

```
            for (int r = 0; r < n; r++) board[r][queens[r]] = 'Q';
            result.push_back(board);
            return;
        }
        for (int c = 0; c < n; c++) {
            if (col[c] || diag1[row - c + n] || diag2[row + c]) continue;
            queens[row] = c;
            col[c] = diag1[row-c+n] = diag2[row+c] = true;
            backtrack(row + 1);
            col[c] = diag1[row-c+n] = diag2[row+c] = false;
        }
    };
    backtrack(0);
    return result;
}
```

Complexity

$\mathcal{O}(n!)$ worst case. In practice, much faster due to diagonal pruning.

3.7 Rat in a Maze

Find all paths from top-left to bottom-right in a grid (0 = blocked, 1 = open).

```
string dirs = "DLRU";
int dr[] = {1, 0, 0, -1};  // D, L, R, U
int dc[] = {0, -1, 1, 0};

void ratMaze(vector<vector<int>>& maze, int r, int c, int n,
             string& path, vector<string>& result) {
    if (r == n-1 && c == n-1) { result.push_back(path); return; }
    maze[r][c] = 0;                    // mark visited (block)
    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d], nc = c + dc[d];
        if (nr >= 0 && nr < n && nc >= 0 && nc < n && maze[nr][nc] == 1) {
            path += dirs[d];
            ratMaze(maze, nr, nc, n, path, result);
            path.pop_back();
        }
    }
    maze[r][c] = 1;                    // unmark (backtrack)
}
// Call: ratMaze(maze, 0, 0, n, path, result)
```

3.8 Knight's Tour

Visit every cell of an $n \times n$ chessboard exactly once with a chess knight.

```
int dx[] = {2,1,-1,-2,-2,-1,1,2};
int dy[] = {1,2,2,1,-1,-2,-2,-1};

bool knightTour(vector<vector<int>>& board, int x, int y, int move, int n) {
    if (move == n * n + 1) return true;      // all cells visited
    for (int d = 0; d < 8; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx >= 0 && nx < n && ny >= 0 && ny < n && board[nx][ny] == 0) {
            board[nx][ny] = move;
            if (knightTour(board, nx, ny, move + 1, n)) return true;
            board[nx][ny] = 0;               // backtrack
        }
    }
    return false;
```

```
}
// Init: board[0][0] = 1; knightTour(board, 0, 0, 2, n)
```

Key Insight / Optimization

Warnsdorff's heuristic: At each step, move to the square with the fewest onward moves. This greedy heuristic makes Knight's Tour run nearly in $\mathcal{O}(n^2)$ for most board sizes and dramatically reduces backtracking.

3.9 Palindrome Partitioning (LeetCode 131)

Partition a string so every substring is a palindrome.

```
bool isPalin(const string& s, int l, int r) {
    while (l < r) if (s[l++] != s[r--]) return false;
    return true;
}

void palindromePartition(const string& s, int start, vector<string>& curr,
                         vector<vector<string>>& result) {
    if (start == (int)s.size()) { result.push_back(curr); return; }
    for (int end = start; end < (int)s.size(); end++) {
        if (isPalin(s, start, end)) {
            curr.push_back(s.substr(start, end - start + 1));
            palindromePartition(s, end + 1, curr, result);
            curr.pop_back();
        }
    }
}
```

3.10 Restore IP Addresses (LeetCode 93)

```
void restoreIp(const string& s, int start, int part, string& curr,
               vector<string>& result) {
    if (part == 4 && start == (int)s.size()) { result.push_back(curr); return; }
    if (part == 4 || start == (int)s.size()) return;

    string saved = curr;
    for (int len = 1; len <= 3 && start + len <= (int)s.size(); len++) {
        string seg = s.substr(start, len);
        if (len > 1 && seg[0] == '0') break;        // leading zero
        if (stoi(seg) > 255) break;                  // exceeds 255
        curr += (part > 0 ? "." : "") + seg;
        restoreIp(s, start + len, part + 1, curr, result);
        curr = saved;
    }
}
// Call: restoreIp(s, 0, 0, curr, result)
```

3.11 Sudoku Solver

```
bool isValidSudoku(vector<vector<char>>& board, int row, int col, char c) {
    for (int i = 0; i < 9; i++) {
        if (board[row][i] == c) return false;
        if (board[i][col] == c) return false;
        int br = 3*(row/3) + i/3, bc = 3*(col/3) + i%3;
        if (board[br][bc] == c) return false;
    }
    return true;
}
```

```
10
11 bool solveSudoku(vector<vector<char>>& board) {
12     for (int r = 0; r < 9; r++) {
13         for (int c = 0; c < 9; c++) {
14             if (board[r][c] != '.') continue;
15             for (char d = '1'; d <= '9'; d++) {
16                 if (!isValidSudoku(board, r, c, d)) continue;
17                 board[r][c] = d;
18                 if (solveSudoku(board)) return true;
19                 board[r][c] = '.';
20             }
21             return false;
22         }
23     }
24     return true;
25 }
```

3.12 Word Search (LeetCode 79)

```
1  bool dfsWord(vector<vector<char>>& board, const string& word, int i, int r, int c) {
2      if (i == (int)word.size()) return true;
3      if (r < 0 || r >= (int)board.size() ||
4          c < 0 || c >= (int)board[0].size()) return false;
5      if (board[r][c] != word[i]) return false;
6
7      char tmp = board[r][c];
8      board[r][c] = '#';                    // mark visited
9      bool found = dfsWord(board, word, i+1, r+1, c)
10                 || dfsWord(board, word, i+1, r-1, c)
11                 || dfsWord(board, word, i+1, r, c+1)
12                 || dfsWord(board, word, i+1, r, c-1);
13     board[r][c] = tmp;                    // restore
14     return found;
15 }
16
17 bool exist(vector<vector<char>>& board, string word) {
18     for (int r = 0; r < (int)board.size(); r++)
19         for (int c = 0; c < (int)board[0].size(); c++)
20             if (dfsWord(board, word, 0, r, c)) return true;
21     return false;
22 }
```

Common Trap / Gotcha

In-place marking: Replace the cell with `'#'` before recursing, restore after. Never forget to restore, or the backtracking is broken.

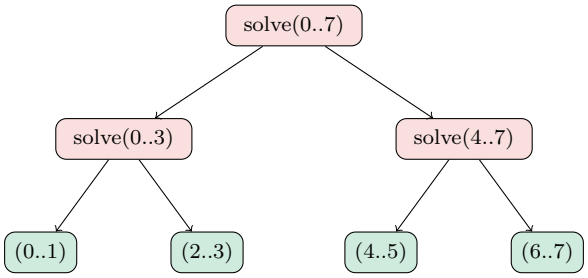
Chapter 4

Divide and Conquer

4.1 Divide and Conquer Template

Pattern / Core Idea

Divide: Split the problem into k subproblems of roughly equal size n/k.
Conquer: Solve each subproblem recursively.
Combine: Merge the results.
Master Theorem: $T(n) = aT(n/b) + f(n)$ where a = subproblems, b = size reduction, $f(n)$ = combine cost.



$\mathcal{O}(\log n)$ levels, $\mathcal{O}(n)$ combine at each level

4.2 Merge Sort

```cpp
void merge(vector<int>& arr, int l, int mid, int r) {
    vector<int> tmp;
    int i = l, j = mid + 1;
    while (i <= mid && j <= r)
        tmp.push_back(arr[i] <= arr[j] ? arr[i++] : arr[j++]);
    while (i <= mid) tmp.push_back(arr[i++]);
    while (j <= r)   tmp.push_back(arr[j++]);
    for (int k = l; k <= r; k++) arr[k] = tmp[k - l];
}

void mergeSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(arr, l, mid);
    mergeSort(arr, mid + 1, r);
    merge(arr, l, mid, r);
}
```

Complexity

$T(n) = 2T(n/2) + \mathcal{O}(n) \Rightarrow \mathcal{O}(n \log n)$ time. $\mathcal{O}(n)$ auxiliary space.

4.2.1 Count Inversions (Merge Sort Application)

An inversion is a pair (i, j) where $i < j$ but $\texttt{arr}[i] > \texttt{arr}[j]$.

```
long long mergeCount(vector<int>& arr, int l, int mid, int r) {
    long long inv = 0;
    vector<int> tmp;
    int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (arr[i] <= arr[j]) { tmp.push_back(arr[i++]); }
        else {
            inv += (mid - i + 1);    // arr[i..mid] all > arr[j]
            tmp.push_back(arr[j++]);
        }
    }
    while (i <= mid) tmp.push_back(arr[i++]);
    while (j <= r)   tmp.push_back(arr[j++]);
    for (int k = l; k <= r; k++) arr[k] = tmp[k - l];
    return inv;
}

long long countInversions(vector<int>& arr, int l, int r) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    return countInversions(arr, l, mid)
         + countInversions(arr, mid + 1, r)
         + mergeCount(arr, l, mid, r);
}
```

4.2.2 Count Reverse Pairs (LeetCode 493)

A reverse pair is (i, j) where $i < j$ and $\texttt{arr}[i] > 2 \cdot \texttt{arr}[j]$.

```
long long mergeRevPairs(vector<int>& arr, int l, int mid, int r) {
    long long cnt = 0;
    int j = mid + 1;
    // Count pairs BEFORE merging (important: do this step first)
    for (int i = l; i <= mid; i++) {
        while (j <= r && (long long)arr[i] > 2LL * arr[j]) j++;
        cnt += (j - mid - 1);
    }
    // Now do regular merge
    vector<int> tmp;
    int p = l, q = mid + 1;
    while (p <= mid && q <= r)
        tmp.push_back(arr[p] <= arr[q] ? arr[p++] : arr[q++]);
    while (p <= mid) tmp.push_back(arr[p++]);
    while (q <= r)   tmp.push_back(arr[q++]);
    for (int k = l; k <= r; k++) arr[k] = tmp[k - l];
    return cnt;
}

long long reversePairs(vector<int>& arr, int l, int r) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    return reversePairs(arr, l, mid)
         + reversePairs(arr, mid + 1, r)
         + mergeRevPairs(arr, l, mid, r);
}
```

4.3 Quick Sort and Variants

4.3.1 Standard Quick Sort

```cpp
int partition(vector<int>& arr, int l, int r) {
    int pivot = arr[r], i = l - 1;
    for (int j = l; j < r; j++)
        if (arr[j] <= pivot) swap(arr[++i], arr[j]);
    swap(arr[i + 1], arr[r]);
    return i + 1;
}

void quickSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int p = partition(arr, l, r);
    quickSort(arr, l, p - 1);
    quickSort(arr, p + 1, r);
}
```

Complexity

Average: $\mathcal{O}(n \log n)$. Worst (sorted input): $\mathcal{O}(n^2)$.

Common Trap / Gotcha

Worst case on sorted input: Always picking the last element as pivot on sorted data gives $\mathcal{O}(n^2)$. Fix: use `random_shuffle` before sorting, or pick a random pivot.

4.3.2 Quick Select (K-th Smallest in O(n) Average)

```cpp
int quickSelect(vector<int>& arr, int l, int r, int k) {
    if (l == r) return arr[l];
    int p = partition(arr, l, r);
    if (p == k)      return arr[p];
    else if (k < p) return quickSelect(arr, l, p - 1, k);
    else            return quickSelect(arr, p + 1, r, k);
}
// k-th smallest (0-indexed): quickSelect(arr, 0, n-1, k)
```

4.3.3 3-Way Partition Quick Sort (Dutch National Flag)

Efficient for arrays with many duplicates.

```cpp
void threeWayPartition(vector<int>& arr, int l, int r, int& lt, int& gt) {
    int pivot = arr[l], i = l + 1;
    lt = l; gt = r;
    while (i <= gt) {
        if (arr[i] < pivot)      swap(arr[lt++], arr[i++]);
        else if (arr[i] > pivot) swap(arr[i], arr[gt--]);
        else                     i++;
    }
}

void quickSort3Way(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int lt, gt;
    threeWayPartition(arr, l, r, lt, gt);
    quickSort3Way(arr, l, lt - 1);
    quickSort3Way(arr, gt + 1, r);
}
```

4.4 Maximum Subarray (Divide and Conquer)

```cpp
int maxCrossing(vector<int>& arr, int l, int mid, int r) {
    int leftSum = INT_MIN, sum = 0;
    for (int i = mid; i >= l; i--) { sum += arr[i]; leftSum = max(leftSum, sum); }
    int rightSum = INT_MIN; sum = 0;
    for (int i = mid + 1; i <= r; i++) { sum += arr[i]; rightSum = max(rightSum, sum); }
    return leftSum + rightSum;
}

int maxSubarray(vector<int>& arr, int l, int r) {
    if (l == r) return arr[l];
    int mid = l + (r - l) / 2;
    return max({maxSubarray(arr, l, mid),
                maxSubarray(arr, mid + 1, r),
                maxCrossing(arr, l, mid, r)});
}
```

Complexity

$\mathcal{O}(n \log n)$ via D&C vs $\mathcal{O}(n)$ with Kadane's algorithm. But D&C builds core intuition for crossing-boundary problems.

4.5 Closest Pair of Points

Find the two closest points from n points in 2D. Naive: $\mathcal{O}(n^2)$. D&C: $\mathcal{O}(n \log n)$.

```cpp
struct Point { double x, y; };

double dist(Point& a, Point& b) {
    return sqrt((a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y));
}

double stripMin(vector<Point>& strip, double d) {
    sort(strip.begin(), strip.end(), [](Point& a, Point& b){ return a.y < b.y; });
    double minD = d;
    for (int i = 0; i < (int)strip.size(); i++)
        for (int j = i+1; j < (int)strip.size() && strip[j].y-strip[i].y < minD; j++)
            minD = min(minD, dist(strip[i], strip[j]));
    return minD;
}

// Points must be sorted by x before calling
double closestPair(vector<Point>& pts, int l, int r) {
    if (r - l <= 2) {                          // base: brute force small cases
        double d = 1e18;
        for (int i = l; i <= r; i++)
            for (int j = i+1; j <= r; j++) d = min(d, dist(pts[i], pts[j]));
        sort(pts.begin()+l, pts.begin()+r+1, [](Point& a, Point& b){ return a.y<b.y;});
        return d;
    }
    int mid = (l + r) / 2;
    double midX = pts[mid].x;
    double d = min(closestPair(pts, l, mid), closestPair(pts, mid+1, r));

    // Merge by y (like merge sort step)
    vector<Point> merged(pts.begin()+l, pts.begin()+r+1);
    inplace_merge(merged.begin(), merged.begin()+(mid-l+1), merged.end(),
                  [](Point& a, Point& b){ return a.y < b.y; });
    copy(merged.begin(), merged.end(), pts.begin()+l);

    // Check strip: points within d of the dividing line
    vector<Point> strip;
```

```
    for (int i = l; i <= r; i++)
        if (abs(pts[i].x - midX) < d) strip.push_back(pts[i]);
    return min(d, stripMin(strip, d));
}
```

Complexity

$\mathcal{O}(n \log^2 n)$ naive, $\mathcal{O}(n \log n)$ with merge-sort-like approach. The strip check only compares at most 8 points per point (geometry proof), so strip checking is $\mathcal{O}(n)$.

4.6 Karatsuba Multiplication

Multiply two n-digit numbers in $\mathcal{O}(n^{1.585})$ instead of $\mathcal{O}(n^2)$.

```
// Multiply two large numbers represented as long long (works for small n)
long long karatsuba(long long x, long long y) {
    if (x < 10 || y < 10) return x * y;        // base case

    int n = max(to_string(x).size(), to_string(y).size());
    int half = n / 2;
    long long p = (long long)pow(10, half);

    long long a = x / p, b = x % p;            // x = a * 10^half + b
    long long c = y / p, d = y % p;            // y = c * 10^half + d

    long long ac = karatsuba(a, c);
    long long bd = karatsuba(b, d);
    long long ad_bc = karatsuba(a + b, c + d) - ac - bd;  // Karatsuba's trick

    return ac * p * p + ad_bc * p + bd;
    // Instead of 4 multiplications (ac, ad, bc, bd), only 3: ac, bd, (a+b)(c+d)
}
```

Key Insight / Optimization

Karatsuba's trick: $(a+b)(c+d) = ac + ad + bc + bd$, so $ad + bc = (a+b)(c+d) - ac - bd$. This saves one recursive multiplication: $T(n) = 3T(n/2) + \mathcal{O}(n) \Rightarrow \mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585})$.

4.7 Master Theorem — Complete Guide

4.7.1 The Recurrence Formula

Pattern / Core Idea

The **Master Theorem** gives a closed-form time complexity for any divide-and-conquer recurrence of the form:

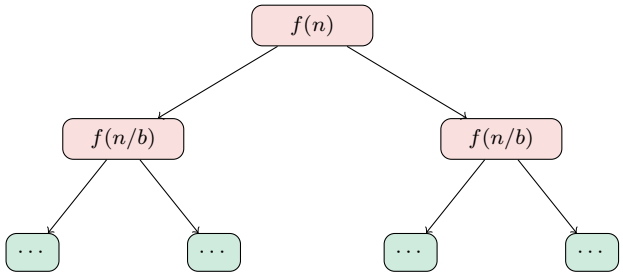
$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where:

- $a \geq 1$ — number of subproblems the problem is split into
- $b > 1$ — factor by which the input size is reduced per subproblem
- $f(n)$ — cost of the work done *outside* the recursive calls (divide + combine)
- $T(1) = \mathcal{O}(1)$ — base case (constant work on a single element)

4.7.2 Recursion Tree Intuition

The tree has $\log_b n$ levels. At level k, there are a^k subproblems each of size n/b^k. The work at each level is $a^k \cdot f(n/b^k)$. Summing across all levels gives three cases:



The **critical exponent** $c^* = \log_b a$ determines which level dominates:

Who dominates?	Intuition
Leaves dominate	Subproblems grow fast (a is large): work piles up at the bottom
Every level equal	Each level contributes equally: multiply by $\log n$ levels
Root dominates	Combine step is expensive: most work is at the top

4.7.3 The Three Cases

Write $f(n) = \mathcal{O}(n^c)$ for some c and compare with $c^* = \log_b a$:

Case 1 — Leaves dominate ($c < c^*$, i.e., $f(n)$ grows slower than n^{c^*}):

$$T(n) = \mathcal{O}(n^{\log_b a})$$

The number of leaves ($= a^{\log_b n} = n^{\log_b a}$) is the bottleneck.

Case 2 — All levels equal ($c = c^*$, i.e., $f(n) = \Theta(n^{c^*})$):

$$T(n) = \mathcal{O}(n^{\log_b a} \cdot \log n)$$

Each of the $\log_b n$ levels contributes $\Theta(n^{c^*})$ work; multiply them.

Case 3 — Root dominates ($c > c^*$, i.e., $f(n)$ grows faster than n^{c^*}):

$$T(n) = \mathcal{O}(f(n))$$

The combine step at the root dwarfs all recursive work. (Also requires the *regularity condition*: $a \cdot f(n/b) \leq k \cdot f(n)$ for some $k < 1$, which holds for all polynomial f.)

Key Insight / Optimization

One-line recipe: Compute $c^* = \log_b a$. Compare $f(n)$ against n^{c^*}:

$$T(n) = \begin{cases} \mathcal{O}(n^{c^*}) & f(n) = \mathcal{O}(n^{c^*-\varepsilon}) \text{ (Case 1)} \\ \mathcal{O}(n^{c^*} \log n) & f(n) = \Theta(n^{c^*}) \text{ (Case 2)} \\ \mathcal{O}(f(n)) & f(n) = \Omega(n^{c^*+\varepsilon}) \text{ (Case 3)} \end{cases}$$

4.7.4 How to Apply It (Step by Step)

Given a recurrence, follow these 4 steps:

1. **Identify** a, b, $f(n)$ from the recurrence $T(n) = a\,T(n/b) + f(n)$.
2. **Compute** $c^* = \log_b a$. This is the "boundary" exponent.
3. **Write** $f(n) = \Theta(n^c)$ and compare c with c^*.
4. **Apply the matching case** to read off the answer.

Example walkthrough — Merge Sort: $T(n) = 2T(n/2) + \Theta(n)$

- Step 1: $a = 2$, $b = 2$, $f(n) = n$.
- Step 2: $c^* = \log_2 2 = 1$.
- Step 3: $f(n) = n = n^1$, so $c = 1 = c^*$.
- Step 4: Case 2 $\Rightarrow T(n) = \mathcal{O}(n^1 \log n) = \mathcal{O}(n \log n)$. ✓

Example walkthrough — Binary Search: $T(n) = T(n/2) + \Theta(1)$

- Step 1: $a = 1$, $b = 2$, $f(n) = 1$.
- Step 2: $c^* = \log_2 1 = 0$.
- Step 3: $f(n) = 1 = n^0$, so $c = 0 = c^*$.
- Step 4: Case 2 $\Rightarrow T(n) = \mathcal{O}(n^0 \log n) = \mathcal{O}(\log n)$. ✓

4.7.5 Extended Case 2 (Logarithmic f)

When $f(n) = \Theta(n^{c^*} \log^k n)$ for some $k \geq 0$, Case 2 extends to:

$$T(n) = \mathcal{O}(n^{c^*} \log^{k+1} n)$$

Example: $T(n) = 2T(n/2) + n \log n \quad (a = 2, b = 2, c^* = 1, f(n) = n \log^1 n)$

$$\Rightarrow \quad T(n) = \mathcal{O}(n \log^2 n)$$

4.7.6 Worked Examples Reference Table

Recurrence	a	b	$f(n)$	$c^* = \log_b a$	**Result**
Binary search	1	2	1	0	$\mathcal{O}(\log n)$ [Case 2]
Merge sort	2	2	n	1	$\mathcal{O}(n \log n)$ [Case 2]
Quick sort (avg)	2	2	n	1	$\mathcal{O}(n \log n)$ [Case 2]
Naive Fibonacci	2	1	—	—	*Master Thm not applicable* ($b = 1$)
Karatsuba mult.	3	2	n	1.585	$\mathcal{O}(n^{1.585})$ [Case 1]
Strassen matrix	7	2	n^2	2.807	$\mathcal{O}(n^{2.807})$ [Case 1]
Naive matrix mult	8	2	n^2	3	$\mathcal{O}(n^3)$ [Case 1]
Max subarray D&C	2	2	n	1	$\mathcal{O}(n \log n)$ [Case 2]
4-peg Hanoi (F-S)	2	2	$\sqrt{n}$	1	$\mathcal{O}(n)$ [Case 3: $f \ll n^{c^*}$]
$T = 4T(n/2) + n$	4	2	n	2	$\mathcal{O}(n^2)$ [Case 1]
$T = 4T(n/2) + n^2$	4	2	n^2	2	$\mathcal{O}(n^2 \log n)$ [Case 2]
$T = 4T(n/2) + n^3$	4	2	n^3	2	$\mathcal{O}(n^3)$ [Case 3]

Common Trap / Gotcha

When the Master Theorem does NOT apply:

- $b \leq 1$: subproblem size doesn't decrease (e.g., naive Fibonacci $T(n) = T(n-1) + T(n-2)$).
- $a < 1$: fractional number of subproblems (meaningless).
- $f(n)$ is not polynomially related to n^{c^*} (e.g., $f(n) = 2^n$).
- Subproblems are of different sizes (use Akra-Bazzi instead).

For recurrences like $T(n) = T(n-1) + \mathcal{O}(1)$, use **substitution** or **telescoping**: this gives $T(n) = \mathcal{O}(n)$ trivially.

Chapter 5

Recursion on Trees & Graphs

5.1 Tree Traversals

Pattern / Core Idea

A tree is inherently recursive: every subtree is itself a tree. The three classic traversals differ only in *when* the root is processed relative to its children.

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};

void inorder(TreeNode* r)  { if(!r) return; inorder(r->left); cout<<r->val<<" "; inorder(r->right); }
void preorder(TreeNode* r) { if(!r) return; cout<<r->val<<" "; preorder(r->left); preorder(r->right); }
void postorder(TreeNode* r){ if(!r) return; postorder(r->left); postorder(r->right); cout<<r->val<<" "; }
```

5.2 Height, Diameter, and Balance

5.2.1 Height and Size

```
int height(TreeNode* r) {
    if (!r) return -1;
    return 1 + max(height(r->left), height(r->right));
}
int size(TreeNode* r) {
    if (!r) return 0;
    return 1 + size(r->left) + size(r->right);
}
```

5.2.2 Diameter of Binary Tree (LeetCode 543)

The diameter is the longest path between any two nodes (may not pass through root).

```
int diameter(TreeNode* root) {
    int diam = 0;
    function<int(TreeNode*)> height = [&](TreeNode* node) -> int {
        if (!node) return -1;
        int lh = height(node->left), rh = height(node->right);
        diam = max(diam, lh + rh + 2);   // path through this node
        return 1 + max(lh, rh);
```

```
    };
    height(root);
    return diam;
}
```

5.2.3 Check if Tree is Height-Balanced (LeetCode 110)

```
// Returns -1 if unbalanced, otherwise returns height
int checkBalanced(TreeNode* node) {
    if (!node) return 0;
    int lh = checkBalanced(node->left);
    if (lh == -1) return -1;
    int rh = checkBalanced(node->right);
    if (rh == -1) return -1;
    if (abs(lh - rh) > 1) return -1;   // unbalanced at this node
    return 1 + max(lh, rh);
}
bool isBalanced(TreeNode* root) { return checkBalanced(root) != -1; }
```

Key Insight / Optimization

Why return -1? We use the return value to carry two pieces of information: whether subtree is balanced AND its height. This avoids a second pass, keeping the solution $\mathcal{O}(n)$ instead of $\mathcal{O}(n^2)$.

5.3 Lowest Common Ancestor (LCA)

```
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* left  = lca(root->left, p, q);
    TreeNode* right = lca(root->right, p, q);
    if (left && right) return root;   // p and q in different subtrees
    return left ? left : right;
}
```

5.3.1 LCA in BST (Optimized)

In a BST, we can use the ordering property to navigate directly:

```
TreeNode* lcaBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root) return nullptr;
    if (p->val < root->val && q->val < root->val)
        return lcaBST(root->left, p, q);    // both in left subtree
    if (p->val > root->val && q->val > root->val)
        return lcaBST(root->right, p, q);   // both in right subtree
    return root;                             // split point = LCA
}
```

5.4 Path Sum Problems

5.4.1 Has Path Sum Root-to-Leaf (LeetCode 112)

```
bool hasPathSum(TreeNode* root, int target) {
    if (!root) return false;
    if (!root->left && !root->right) return root->val == target;
    return hasPathSum(root->left,  target - root->val)
        || hasPathSum(root->right, target - root->val);
}
```

5.4.2 All Root-to-Leaf Paths with Sum (LeetCode 113)

```
void pathSum(TreeNode* node, int target, vector<int>& curr,
             vector<vector<int>>& result) {
    if (!node) return;
    curr.push_back(node->val);
    if (!node->left && !node->right && node->val == target)
        result.push_back(curr);
    pathSum(node->left,  target - node->val, curr, result);
    pathSum(node->right, target - node->val, curr, result);
    curr.pop_back();                        // backtrack
}
```

5.4.3 Maximum Path Sum (Any Path, LeetCode 124)

```
int maxPathSum(TreeNode* root) {
    int globalMax = INT_MIN;
    function<int(TreeNode*)> dfs = [&](TreeNode* node) -> int {
        if (!node) return 0;
        int leftGain  = max(0, dfs(node->left));
        int rightGain = max(0, dfs(node->right));
        globalMax = max(globalMax, node->val + leftGain + rightGain);
        return node->val + max(leftGain, rightGain);  // only one arm upward
    };
    dfs(root);
    return globalMax;
}
```

5.5 BST Operations

5.5.1 Search in BST

```
TreeNode* searchBST(TreeNode* root, int target) {
    if (!root || root->val == target) return root;
    if (target < root->val) return searchBST(root->left, target);
    return searchBST(root->right, target);
}
```

5.5.2 Insert into BST

```
TreeNode* insertBST(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);   // insert here
    if (val < root->val) root->left  = insertBST(root->left,  val);
    else                 root->right = insertBST(root->right, val);
    return root;
}
```

5.5.3 Delete from BST

```
TreeNode* deleteBST(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) { root->left  = deleteBST(root->left,  key); return root; }
    if (key > root->val) { root->right = deleteBST(root->right, key); return root; }
    // key == root->val: this node must be deleted
    if (!root->left)  return root->right;
    if (!root->right) return root->left;
    // Two children: find in-order successor (smallest in right subtree)
    TreeNode* succ = root->right;
    while (succ->left) succ = succ->left;
    root->val = succ->val;
    root->right = deleteBST(root->right, succ->val);
    return root;
```

```
}
```

5.5.4 Validate BST (LeetCode 98)

```
bool validateBST(TreeNode* node, long long lo, long long hi) {
    if (!node) return true;
    if (node->val <= lo || node->val >= hi) return false;
    return validateBST(node->left,  lo, node->val)
        && validateBST(node->right, node->val, hi);
}
bool isValidBST(TreeNode* root) {
    return validateBST(root, LLONG_MIN, LLONG_MAX);
}
```

5.6 Tree Construction

5.6.1 Build Tree from Preorder + Inorder (LeetCode 105)

```
TreeNode* buildTree(vector<int>& pre, vector<int>& in,
                    int preL, int preR, int inL, int inR,
                    unordered_map<int,int>& inIdx) {
    if (preL > preR) return nullptr;
    int rootVal = pre[preL];
    TreeNode* root = new TreeNode(rootVal);
    int inMid = inIdx[rootVal], leftSize = inMid - inL;
    root->left  = buildTree(pre, in, preL+1, preL+leftSize, inL, inMid-1, inIdx);
    root->right = buildTree(pre, in, preL+leftSize+1, preR, inMid+1, inR, inIdx);
    return root;
}
```

5.6.2 Serialize and Deserialize (LeetCode 297)

```
string serialize(TreeNode* root) {
    if (!root) return "N,";
    return to_string(root->val) + "," + serialize(root->left) + serialize(root->right);
}

TreeNode* deserialize(const string& data, int& i) {
    if (data[i] == 'N') { i += 2; return nullptr; } // skip "N,"
    int sign = 1;
    if (data[i] == '-') { sign = -1; i++; }
    int val = 0;
    while (i < (int)data.size() && data[i] != ',') val = val*10 + (data[i++]-'0');
    i++;  // skip ','
    TreeNode* node = new TreeNode(sign * val);
    node->left  = deserialize(data, i);
    node->right = deserialize(data, i);
    return node;
}
```

5.7 Graph DFS (Recursive)

5.7.1 Detect Cycle in Directed Graph

```
bool dfsCycle(int u, vector<vector<int>>& adj,
              vector<bool>& vis, vector<bool>& inStack) {
    vis[u] = inStack[u] = true;
    for (int v : adj[u]) {
        if (!vis[v] && dfsCycle(v, adj, vis, inStack)) return true;
        if (inStack[v]) return true;
```

```
    }
    inStack[u] = false;
    return false;
}
```

5.7.2 Topological Sort (DFS)

```
void topoSort(int u, vector<vector<int>>& adj,
              vector<bool>& vis, stack<int>& stk) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) topoSort(v, adj, vis, stk);
    stk.push(u);    // push AFTER all descendants (post-order)
}

vector<int> topologicalSort(int n, vector<vector<int>>& adj) {
    vector<bool> vis(n, false);
    stack<int> stk;
    for (int i = 0; i < n; i++) if (!vis[i]) topoSort(i, adj, vis, stk);
    vector<int> order;
    while (!stk.empty()) { order.push_back(stk.top()); stk.pop(); }
    return order;
}
```

5.7.3 Number of Islands (DFS Flood Fill)

```
void dfsIsland(vector<vector<char>>& grid, int r, int c) {
    if (r < 0 || r >= (int)grid.size() ||
        c < 0 || c >= (int)grid[0].size() || grid[r][c] != '1') return;
    grid[r][c] = '0';
    dfsIsland(grid, r+1, c); dfsIsland(grid, r-1, c);
    dfsIsland(grid, r, c+1); dfsIsland(grid, r, c-1);
}

int numIslands(vector<vector<char>>& grid) {
    int count = 0;
    for (int r = 0; r < (int)grid.size(); r++)
        for (int c = 0; c < (int)grid[0].size(); c++)
            if (grid[r][c] == '1') { dfsIsland(grid, r, c); count++; }
    return count;
}
```

Common Trap / Gotcha

Stack overflow on large grids: Recursive DFS can overflow for grids $> 10^4 \times 10^4$. Switch to iterative BFS/DFS with an explicit stack for such constraints.

Chapter 6

Advanced Recursion Patterns

6.1 Memoization — From Naive to Optimal

Pattern / Core Idea

Top-down (memoization): Recursive + cache. Natural to write, starts from the original problem, only computes needed states.
Bottom-up (tabulation): Iterative DP. No stack overhead, often faster.
Rule: When two different call paths reach the same (arguments), memoize.

6.1.1 Longest Common Subsequence

```
int dp[1001][1001];

int lcs(const string& a, const string& b, int i, int j) {
    if (i == 0 || j == 0) return 0;
    if (dp[i][j] != -1) return dp[i][j];
    if (a[i-1] == b[j-1]) return dp[i][j] = 1 + lcs(a, b, i-1, j-1);
    return dp[i][j] = max(lcs(a, b, i-1, j), lcs(a, b, i, j-1));
}
```

6.1.2 Edit Distance (Levenshtein)

```
int editDist[1001][1001];

int edit(const string& a, const string& b, int i, int j) {
    if (i == 0) return j;                          // insert j characters
    if (j == 0) return i;                          // delete i characters
    if (editDist[i][j] != -1) return editDist[i][j];
    if (a[i-1] == b[j-1]) return editDist[i][j] = edit(a, b, i-1, j-1);
    return editDist[i][j] = 1 + min({edit(a, b, i-1, j),    // delete from a
                                     edit(a, b, i, j-1),    // insert into a
                                     edit(a, b, i-1, j-1)   // replace
                                    });
}
```

6.2 Regular Expression Matching (LeetCode 10)

Match string s against pattern p where `.` matches any character and `*` matches zero or more of the preceding element.

```
int memo[10001][10001];

```

```cpp
bool isMatch(const string& s, const string& p, int i, int j) {
    if (j == (int)p.size()) return i == (int)s.size();  // pattern exhausted
    if (memo[i][j] != -1) return memo[i][j];

    bool firstMatch = (i < (int)s.size() && (p[j] == s[i] || p[j] == '.'));

    if (j + 1 < (int)p.size() && p[j+1] == '*') {
        // Option 1: skip the 'x*' pattern (zero occurrences)
        // Option 2: match one char and stay at 'x*'
        return memo[i][j] = isMatch(s, p, i, j+2) ||
                            (firstMatch && isMatch(s, p, i+1, j));
    }
    return memo[i][j] = firstMatch && isMatch(s, p, i+1, j+1);
}
```

Complexity

$\mathcal{O}(|s| \cdot |p|)$ time and space with memoization.

6.3 Wildcard Matching (LeetCode 44)

Match s against p where `?` matches any single character and `*` matches any sequence (including empty).

```cpp
int wldMemo[10001][10001];

bool wildcard(const string& s, const string& p, int i, int j) {
    if (i == (int)s.size() && j == (int)p.size()) return true;
    if (j == (int)p.size()) return false;
    if (wldMemo[i][j] != -1) return wldMemo[i][j];

    if (p[j] == '*') {
        // '*' matches empty: advance j
        // '*' matches one char: advance i (and stay at '*')
        return wldMemo[i][j] = wildcard(s, p, i, j+1) ||
                               (i < (int)s.size() && wildcard(s, p, i+1, j));
    }
    if (i < (int)s.size() && (p[j] == '?' || p[j] == s[i]))
        return wldMemo[i][j] = wildcard(s, p, i+1, j+1);
    return wldMemo[i][j] = false;
}
```

6.4 Decode Ways (LeetCode 91)

```cpp
int numDecodings(string s) {
    int n = s.size();
    vector<int> dp(n + 1, -1);
    function<int(int)> solve = [&](int i) -> int {
        if (i == n) return 1;
        if (s[i] == '0') return 0;
        if (dp[i] != -1) return dp[i];
        int ways = solve(i + 1);
        if (i + 1 < n) {
            int two = stoi(s.substr(i, 2));
            if (two >= 10 && two <= 26) ways += solve(i + 2);
        }
        return dp[i] = ways;
    };
    return solve(0);
}
```

6.5 Generate Expressions (LeetCode 282)

Insert $+$, $-$, $*$ between digits to reach a target.

```
vector<string> addOperators(string num, int target) {
    vector<string> result;
    function<void(int, string, long long, long long)> dfs =
        [&](int pos, string expr, long long eval, long long mult) {
        if (pos == (int)num.size()) {
            if (eval == target) result.push_back(expr);
            return;
        }
        for (int len = 1; pos + len <= (int)num.size(); len++) {
            string part = num.substr(pos, len);
            if (len > 1 && part[0] == '0') break;   // no leading zeros
            long long val = stoll(part);
            if (pos == 0) {
                dfs(len, part, val, val);
            } else {
                dfs(pos+len, expr+"+"+part, eval+val,                  val);
                dfs(pos+len, expr+"-"+part, eval-val,                 -val);
                dfs(pos+len, expr+"*"+part, eval-mult+mult*val, mult*val);
            }
        }
    };
    dfs(0, "", 0, 0);
    return result;
}
```

Key Insight / Optimization

Handling multiplication precedence: Track `mult` (last multiplicand). When you apply $*$: undo last add/sub (`eval - mult`) and add new product (`mult * val`).

6.6 Different Ways to Add Parentheses (LeetCode 241)

```
unordered_map<string, vector<int>> cache;

vector<int> diffWays(string expr) {
    if (cache.count(expr)) return cache[expr];
    vector<int> result;
    for (int i = 0; i < (int)expr.size(); i++) {
        char c = expr[i];
        if (c == '+' || c == '-' || c == '*') {
            vector<int> left  = diffWays(expr.substr(0, i));
            vector<int> right = diffWays(expr.substr(i + 1));
            for (int l : left) for (int r : right) {
                if (c == '+') result.push_back(l + r);
                if (c == '-') result.push_back(l - r);
                if (c == '*') result.push_back(l * r);
            }
        }
    }
    if (result.empty()) result.push_back(stoi(expr)); // pure number
    return cache[expr] = result;
}
```

6.7 Recursion with Bitmask (State Compression)

Pattern / Core Idea

When the "state" is a subset of items (used/visited), represent it as a bitmask. Combine recursion with bitmask to enumerate all 2^n states.
Common use cases: TSP, assignment problems, game states.

6.7.1 Minimum Cost Assignment (Bitmask DP)

```
int n;
int cost[20][20];
int memo2[20][1 << 20];

// person = current person (0..n-1), mask = bitmask of assigned tasks
int assign(int person, int mask) {
    if (person == n) return 0;
    if (memo2[person][mask] != -1) return memo2[person][mask];
    int best = INT_MAX;
    for (int task = 0; task < n; task++) {
        if (mask & (1 << task)) continue;
        int sub = assign(person + 1, mask | (1 << task));
        best = min(best, cost[person][task] + sub);
    }
    return memo2[person][mask] = best;
}
```

6.7.2 Travelling Salesman Problem (Bitmask DP)

```
int dist2[20][20];
int tspMemo[20][1 << 20];

// pos = current city, visited = bitmask of visited cities
int tsp(int pos, int visited, int n) {
    if (visited == (1 << n) - 1) return dist2[pos][0];  // return to start
    if (tspMemo[pos][visited] != -1) return tspMemo[pos][visited];
    int best = INT_MAX;
    for (int next = 0; next < n; next++) {
        if (visited & (1 << next)) continue;              // already visited
        int res = tsp(next, visited | (1 << next), n);
        if (res != INT_MAX)
            best = min(best, dist2[pos][next] + res);
    }
    return tspMemo[pos][visited] = best;
}
// Start at city 0: tsp(0, 1, n)
```

Complexity

$\mathcal{O}(n^2 \cdot 2^n)$ time, $\mathcal{O}(n \cdot 2^n)$ space. Feasible for $n \leq 20$.

6.8 Mutual Recursion (Expression Parser)

```
struct Parser {
    string s; int pos = 0;

    int parseExpr() {           // handles + and -
        int val = parseTerm();
        while (pos < (int)s.size() && (s[pos]=='+' || s[pos]=='-')) {
            char op = s[pos++];
            int r = parseTerm();
```

```
            val = (op == '+') ? val + r : val - r;
        }
        return val;
    }
    int parseTerm() {           // handles * and /
        int val = parseFactor();
        while (pos < (int)s.size() && (s[pos]=='*' || s[pos]=='/')) {
            char op = s[pos++];
            int r = parseFactor();
            val = (op == '*') ? val * r : val / r;
        }
        return val;
    }
    int parseFactor() {         // handles numbers and parentheses
        if (s[pos] == '(') { pos++; int val = parseExpr(); pos++; return val; }
        int val = 0;
        while (pos < (int)s.size() && isdigit(s[pos]))
            val = val * 10 + (s[pos++] - '0');
        return val;
    }
};
// Usage: Parser p; p.s = "3*(2+1)"; cout << p.parseExpr(); // 9
```

6.9 Converting Recursion to Iteration

Pattern / Core Idea

Any recursive function can be converted to iterative using an explicit stack. This eliminates stack-overflow risk and reduces function-call overhead.

```
// Iterative inorder traversal
vector<int> inorderIterative(TreeNode* root) {
    vector<int> result;
    stack<TreeNode*> stk;
    TreeNode* cur = root;
    while (cur || !stk.empty()) {
        while (cur) { stk.push(cur); cur = cur->left; }
        cur = stk.top(); stk.pop();
        result.push_back(cur->val);
        cur = cur->right;
    }
    return result;
}

// Iterative DFS (general graph)
void dfsIterative(int start, vector<vector<int>>& adj) {
    stack<int> stk;
    vector<bool> visited(adj.size(), false);
    stk.push(start);
    while (!stk.empty()) {
        int u = stk.top(); stk.pop();
        if (visited[u]) continue;
        visited[u] = true;
        cout << u << " ";
        for (int v : adj[u]) if (!visited[v]) stk.push(v);
    }
}
```

6.10 Common Recursion Traps

Common Trap / Gotcha

1. Missing base case. Ask: "What is the simplest input? What should I return then?"

Common Trap / Gotcha

2. Not making progress. Every call must move *strictly* toward the base case. Same arguments = infinite recursion.

Common Trap / Gotcha

3. Stack overflow. Default stack $\approx$ 1–8 MB. For $n > 10^5$, prefer iterative or increase stack.

Common Trap / Gotcha

4. Re-computing subproblems. If the same (arguments) are reachable from two different paths, memoize.

Common Trap / Gotcha

5. Forgetting to backtrack. In backtracking, always undo the change after the recursive call. Shared state gets corrupted otherwise.

Common Trap / Gotcha

6. Off-by-one in base cases. Be precise: is it `i == n` or `i == n-1`? Draw small examples to verify.

6.11 Recursion Pattern Summary

Pattern	Structure	Time
Linear recursion	One call per step	$\mathcal{O}(n)$
Binary recursion (D&C)	Two calls, half size	$\mathcal{O}(n \log n)$
Exponential (subsets)	Two calls, same size	$\mathcal{O}(2^n)$
Permutations	n choices at each level	$\mathcal{O}(n!)$
Memoized overlapping	Polynomial states	$\mathcal{O}(n^k)$
Bitmask + recursion	2^n states, n transitions	$\mathcal{O}(n \cdot 2^n)$

Key Insight / Optimization

Decision flowchart:

1. Is the problem naturally recursive? (subproblem structure) $\rightarrow$ Yes: write the recurrence.
2. Are there overlapping subproblems? $\rightarrow$ Yes: add memoization.
3. Is n large enough to cause stack overflow ($> 10^5$)? $\rightarrow$ Yes: convert to iterative.
4. Is it a search/enumeration problem? $\rightarrow$ Use backtracking with pruning.
5. Does the state involve choosing subsets? $\rightarrow$ Consider bitmask DP.